SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	**CO4:** Articulation (BL4, BL5)	**Assessment:**	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	**Total Marks:**	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	A.Ansuha	**Reg. No.:**	192472083
Section / Batch:	CSE - AI	**Date:**	19-08-26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Focus Area	Questions
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Coding Challenge Task

Q3. Traveling Salesman Problem (TSP)

A delivery agent must visit multiple cities exactly once and return to the starting location. The travel costs between each pair of cities are given in a matrix. The goal is to determine the most cost-efficient route that minimizes total travel cost. Use Dynamic Programming (Held-Karp method) to solve the problem efficiently.

Sample Input:

n = 4

Cost =

0 10 15 20

10 0 35 25

15 35 0 30

20 25 30 0

Expected Output:

Minimum Cost = 80

Q3. Traveling Salesman Problem (TSP) – Dynamic Programming

1. Problem Understanding

The Traveling Salesman Problem (TSP) requires a delivery agent to:

- Start from a city.
- Visit every city exactly once.
- Return to the starting city.
- Minimize the total travel cost.

For the given problem, the Held-Karp Dynamic Programming method is used because it avoids checking every possible permutation independently.

2. Objective

1. To find the minimum-cost route for a delivery agent to visit all cities exactly once and return to the starting city.
2. To solve the Traveling Salesman Problem using Dynamic Programming (Held-Karp method).
3. To reduce the computational cost compared with the brute-force approach.
4. To analyze the time and space complexity of the Dynamic Programming approach.
5. To compare the Dynamic Programming solution with a Greedy strategy.

3. Given Input

n = 4

Cost =
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0

4. Dynamic Programming Approach

Define:

dp[mask][i] = minimum cost to visit all cities represented by mask
and finish at city i

where mask represents the set of cities already visited.
For example: mask = 0111

means cities 0, 1, 2 have been visited.
The recurrence is: dp[mask][i] = min(dp[mask without i][j] + cost[j][i])

where j is any previously visited city.

Finally, after visiting all cities, return to city 0:
Answer = min(dp[all_cities][i] + cost[i][0])

5.Algorithm – Dynamic Programming (Held-Karp)

Step 1: Read the number of cities n and the cost matrix.

Step 2: Create a DP table dp[mask][i], where mask represents
the visited cities and i is the current city.

Step 3: Initialize dp[1][0] = 0 because the journey starts
from city 0.

Step 4: For every visited set, try moving from the current city
to every unvisited city.

Step 5: Update the minimum cost:
dp[new_mask][next] =
min(dp[new_mask][next],
dp[mask][current] + cost[current][next])

Step 6: After visiting all cities, add the cost of returning
from the last city to city 0.

Step 7: Find the minimum among all possible final cities.

Step 8: Display the minimum travel cost.

Algorithm – Greedy Strategy

Step 1: Start from city 0.

Step 2: Mark the starting city as visited.

Step 3: Select the nearest unvisited city.

Step 4: Move to that city and add its travel cost.

Step 5: Repeat until all cities are visited.

Step 6: Return to the starting city.

Step 7: Display the total travel cost.

6. Pseudocode

```
SET dp[mask][city] = INF
SET dp[1][0] = 0

FOR each mask
   FOR each current city i
      FOR each next city j
         IF city j is not visited
            UPDATE dp[mask + j][j]
               USING dp[mask][i] + cost[i][j]

ANSWER = minimum dp[all][i] + cost[i][0]
PRINT ANSWER
```

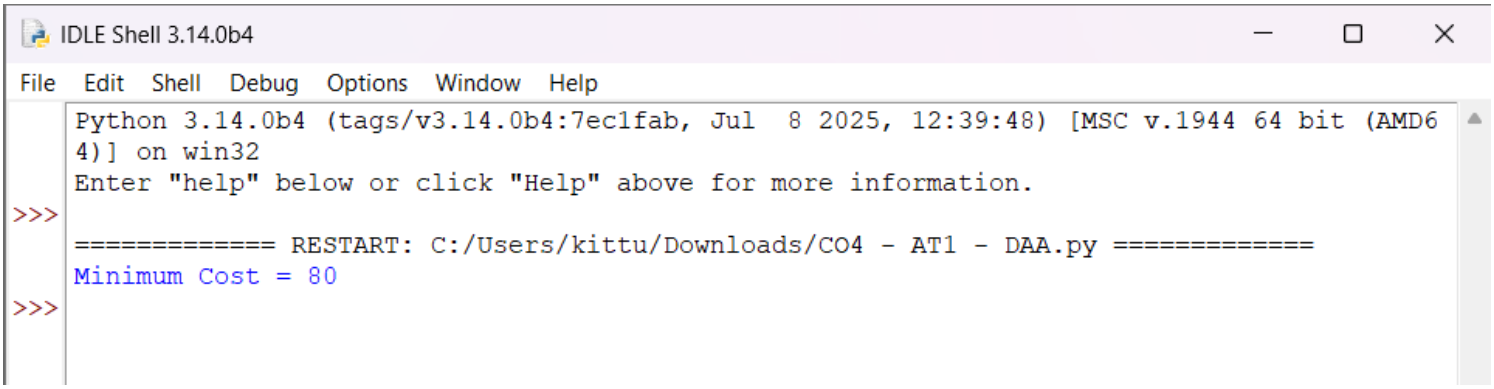
7.Python Code

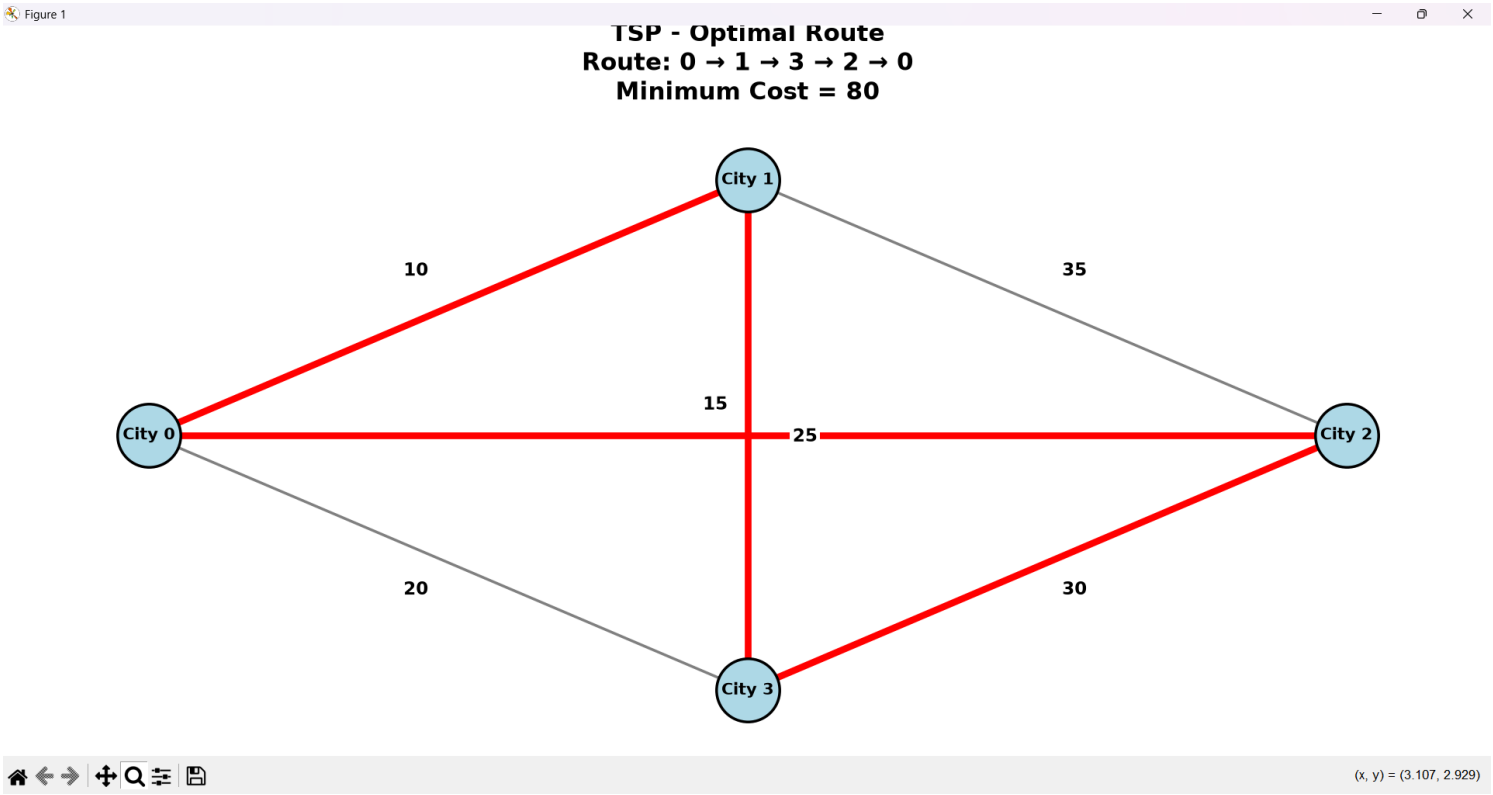
CO4 - AT1 - DAA.py - C:/Users/kittu/Downloads/CO4 - AT1 - DAA.py (3.14.0b4)

File Edit Format Run Options Window Help

```
def tsp(cost):
    n = len(cost)
    INF = float('inf')

    # dp[mask][i] = minimum cost to visit cities in mask
    # and end at city i
    dp = [[INF] * n for _ in range(1 << n)]

    # Start from city 0
    dp[1][0] = 0

    for mask in range(1 << n):
        for current in range(n):

            if dp[mask][current] == INF:
                continue

            # Try visiting an unvisited city
            for next_city in range(n):

                if mask & (1 << next_city) == 0:

                    new_mask = mask | (1 << next_city)

                    dp[new_mask][next_city] = min(
                        dp[new_mask][next_city],
                        dp[mask][current] + cost[current][next_city]
                    )

    # All cities visited
    all_cities = (1 << n) - 1

    # Return to starting city
    answer = INF

    for city in range(1, n):
        answer = min(
            answer,
            dp[all_cities][city] + cost[city][0]
        )

    return answer


# Input
n = 4
```

7.Output





One optimal route is: 0 → 1 → 3 → 2 → 0
Cost: 10 + 15 + 30 + 25 = 80

8. Time Complexity – Dynamic Programming

There are: 2^n
possible subsets of cities.

For every subset, we consider up to n current cities and n next cities.
Therefore: Time Complexity = $O(n^2 \times 2^n)$

Space Complexity

The DP table contains:
$2^n \times n$ states.

Therefore: Space Complexity = $O(n \times 2^n)$
This is much better than checking all possible routes using brute force.

Brute Force:

Time = $O(n!)$
Space = $O(n)$

Held-Karp DP:
Time = $O(n^2 \times 2^n)$
Space = $O(n \times 2^n)$

Approach	Time Complexity	Space Complexity	Optimal?
Held-Karp DP	$O(n^2 \times 2^n)$	$O(n \times 2^n)$	Yes

9.Final Conclusion

For the given TSP problem, Dynamic Programming using the Held-Karp method is the appropriate algorithm because it guarantees the optimal minimum-cost route while significantly reducing the complexity compared with brute force.

Minimum Cost = 80
Optimal Route = $0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$
Time Complexity = $O(n^2 \times 2^n)$
Space Complexity = $O(n \times 2^n)$

10.Manual Solution – TSP Using Dynamic Programming

Given Cost Matrix

	0	1	2	3
0	0	10	15	20
1	10	0	35	25
2	15	35	0	30
3	20	25	30	0

We start from City 0.

Step 1: List Possible Routes

Since City 0 is fixed as the starting city, the remaining cities are:

1, 2, 3

There are:

3! = 6

possible routes.

Step 2: Calculate Each Route Cost

Route 1: 0 → 1 → 2 → 3 → 0

= 10 + 35 + 30 + 20

= 95

Route 2: 0 → 1 → 3 → 2 → 0

= 10 + 25 + 30 + 15

= 80

Route 3: 0 → 2 → 1 → 3 → 0

= 15 + 35 + 25 + 20

= 95

Route 4:0 → 2 → 3 → 1 → 0

= 15 + 30 + 25 + 10

= 80

Route 5:

0 → 3 → 1 → 2 → 0

= 20 + 25 + 35 + 15 = 95

Route 6: 0 → 3 → 2 → 1 → 0
= 20 + 30 + 35 + 10
= 95

Step 3: Compare the Costs

Route	Total Cost
0 → 1 → 2 → 3 → 0	95
0 → 1 → 3 → 2 → 0	80
0 → 2 → 1 → 3 → 0	95
0 → 2 → 3 → 1 → 0	80
0 → 3 → 1 → 2 → 0	95
0 → 3 → 2 → 1 → 0	95

Therefore:
Minimum Cost = 80

There are two equivalent optimal routes:
0 → 1 → 3 → 2 → 0
and
0 → 2 → 3 → 1 → 0

Both have cost: 10 + 25 + 30 + 15 = 80
Final Answer
Optimal Route = 0 → 1 → 3 → 2 → 0
Minimum Cost = 80

Code of Conduct

I _____(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A.Anusha

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%				
Algorithm	25%				
Code Implementation	20%				
Competitive Performance	20%				
Test Case Handling	15%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____